

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE FÍSICA DE SÃO CARLOS
BACHARELADO EM FÍSICA

CAIRO HENRIQUE VAZ COTRIM
No USP: 16814178

PROJETO 1 – INTRODUÇÃO À PROGRAMAÇÃO

7600017 – INTRODUÇÃO À FÍSICA COMPUTACIONAL

SÃO CARLOS – SP
2026

Introdução

Este documento apresenta o relatório do Projeto 1, cujo principal objetivo é propiciar um treinamento inicial em programação utilizando exclusivamente a linguagem FORTRAN-77. Para cada problema exigido, são apresentados uma descrição do fenômeno físico ou matemático envolvido, a estratégia computacional desenvolvida e os resultados obtidos das execuções e compilações próprias.

1 Problema 1: Área e Volume de um Torus

1.1 Descrição do Problema

O problema exige o cálculo da área total e do volume de um torus. O programa deve ler do terminal dois valores reais: o raio interno (r_1) e o raio externo (r_2).

As equações analíticas utilizadas para calcular a área superficial A e o volume V são dadas por

$$A = 4\pi^2 Rr$$

e

$$V = 2\pi^2 Rr^2.$$

1.2 Estratégia de Solução

O programa realiza inicialmente a leitura dos valores dos raios interno e externo fornecidos pelo usuário. As variáveis são declaradas como 'REAL*8', permitindo que os cálculos sejam realizados em dupla precisão. O valor de π é definido por meio de um parâmetro constante.

A partir dos valores fornecidos, a área e o volume são calculados diretamente pelas expressões analíticas. No código, o raio interno r_1 corresponde a r , enquanto o raio externo r_2 corresponde a R . Assim, as expressões implementadas são

$$A = 4\pi^2 r_1 r_2$$

e

$$V = 2\pi^2 r_2 r_1^2.$$

Por fim, os valores calculados são impressos no terminal.

1.3 Resultados Obtidos

Para os valores

$$r_1 = 1 \quad \text{e} \quad r_2 = 2,$$

o programa forneceu os seguintes resultados:

Area do torus: 78.956839603063145

Volume do torus: 39.478419801531572

Os valores podem ser verificados diretamente pelas expressões analíticas. Para a área, tem-se

$$A = 4\pi^2(2)(1) = 8\pi^2 \approx 78.9568396030631.$$

Para o volume,

$$V = 2\pi^2(2)(1)^2 = 4\pi^2 \approx 39.4784198015316.$$

Portanto, os resultados numéricos obtidos pelo programa são consistentes com os valores esperados a partir das expressões analíticas.

2 Problema 2: Prisma de Base Triangular

2.1 Descrição do Problema

O objetivo é calcular a área lateral e o volume de um prisma de base triangular a partir de três vetores $\vec{v}_1$, $\vec{v}_2$ e $\vec{v}_3$ lidos do terminal.

Há duas condições geométricas que devem ser verificadas antes da realização dos cálculos. Primeiramente, os vetores $\vec{v}_1$ e $\vec{v}_2$ não podem ser colineares, pois, nesse caso, não definem uma base triangular. Além disso, o vetor $\vec{v}_3$ não pode pertencer ao mesmo plano definido por $\vec{v}_1$ e $\vec{v}_2$. Caso alguma dessas condições seja violada, o programa informa o erro correspondente e encerra a execução.

2.2 Estratégia de Solução

O programa realiza a leitura das componentes dos três vetores pelo terminal. Para os cálculos, todas as variáveis são declaradas como ‘REAL*8’, permitindo trabalhar com dupla precisão.

A verificação de que $\vec{v}_1$ e $\vec{v}_2$ não são colineares é feita calculando o módulo do produto vetorial entre os dois vetores. Se

$$|\vec{v}_1 \times \vec{v}_2| = 0,$$

os vetores são colineares e o programa é encerrado.

Para verificar se os três vetores são coplanares, calcula-se o produto misto

$$\vec{v}_1 \cdot (\vec{v}_2 \times \vec{v}_3).$$

Se o resultado for zero,

$$\vec{v}_1 \cdot (\vec{v}_2 \times \vec{v}_3) = 0,$$

os três vetores são coplanares e o programa também é encerrado.

Essas verificações são implementadas por meio de estruturas condicionais ‘IF’ do FORTRAN-77. Os cálculos dos produtos vetorial, escalar e misto são separados em sub-rotinas e funções, facilitando a organização do programa.

A área lateral é obtida pela soma das áreas dos três paralelogramos formados pelas arestas laterais paralelas a $\vec{v}_3$. Assim,

$$A_{\text{lat}} = |\vec{v}_1 \times \vec{v}_3| + |\vec{v}_2 \times \vec{v}_3| + |(\vec{v}_1 - \vec{v}_2) \times \vec{v}_3|.$$

O volume do prisma corresponde à metade do volume do paralelepípedo formado pelos três vetores. Portanto,

$$V = \frac{1}{2} |\vec{v}_1 \cdot (\vec{v}_2 \times \vec{v}_3)|.$$

2.3 Resultados Obtidos

Foram realizados três casos de teste para verificar tanto os cálculos quanto as condições de encerramento do programa.

No primeiro caso, foram utilizados

$$\vec{v}_1 = (1, 0, 0), \quad \vec{v}_2 = (0, 1, 0), \quad \vec{v}_3 = (0, 0, 1).$$

Como os três vetores são linearmente independentes, o programa realiza os cálculos e fornece:

Area lateral do prisma: 3.4142135623730949
Volume do prisma: 0.500000000000000000

Para esses vetores,

$$|\vec{v}_1 \times \vec{v}_3| = 1,$$

$$|\vec{v}_2 \times \vec{v}_3| = 1$$

e

$$|(\vec{v}_1 - \vec{v}_2) \times \vec{v}_3| = \sqrt{2}.$$

Logo,

$$A_{\text{lat}} = 1 + 1 + \sqrt{2} \approx 3.414213562373095,$$

em concordância com o resultado fornecido pelo programa.

O produto misto dos três vetores possui módulo igual a 1. Dessa forma,

$$V = \frac{|1|}{2} = 0.5,$$

também em concordância com o resultado numérico.

No segundo caso, foram utilizados

$$\vec{v}_1 = (1, 0, 0), \quad \vec{v}_2 = (0, 1, 0), \quad \vec{v}_3 = (1, 1, 0).$$

Nesse caso, os três vetores pertencem ao plano xy . Consequentemente, o produto misto é nulo. O programa identificou corretamente a coplanaridade e encerrou a execução, apresentando:

`Os vetores v1, v2 e v3 sao coplanares.`

Por fim, no terceiro caso foram utilizados

$$\vec{v}_1 = (1, 0, 0), \quad \vec{v}_2 = (2, 0, 0), \quad \vec{v}_3 = (0, 0, 1).$$

Os vetores $\vec{v}_1$ e $\vec{v}_2$ são colineares, pois possuem a mesma direção. Assim,

$$\vec{v}_1 \times \vec{v}_2 = \vec{0},$$

e o programa identifica a condição inválida antes de realizar os cálculos, apresentando:

`Os vetores v1 e v2 sao colineares.`

Os três testes mostram que o programa realiza corretamente os cálculos para uma configuração geometricamente válida e também identifica as duas situações que impedem a construção do prisma: a colinearidade dos vetores da base e a coplanaridade dos três vetores.

3 Problema 3: Ordenação de Números Reais

3.1 Descrição do Problema

O programa deve ler um arquivo contendo N números reais de dupla precisão ('REAL*8'), determinar o valor de N e imprimi-lo no terminal. Em seguida, deve ser fornecido pelo usuário um valor $M \leq N$.

A tarefa consiste em encontrar e ordenar apenas os M menores números do conjunto, sem a necessidade de ordenar todos os N valores.

3.2 Estratégia de Solução

O programa utiliza um vetor de tamanho fixo para armazenar os números lidos do arquivo, uma vez que a alocação dinâmica de memória não faz parte dos recursos permitidos no FORTRAN-77. Neste caso, foram reservadas 1000000 posições para garantir leitura de arquivos de tamanho razoável.

A leitura do arquivo é realizada com o comando 'READ', utilizando a variável 'IOSTAT' para verificar o término da leitura. Enquanto 'IOSTAT=0', um novo número é armazenado no vetor e o contador N é incrementado. Quando a leitura não é mais possível, o laço é encerrado e o valor de N corresponde à quantidade de números presentes no arquivo.

Em seguida, utiliza-se o comando 'REWIND' para retornar o ponteiro do arquivo ao início. Embora o arquivo não seja novamente percorrido posteriormente no algoritmo, o comando é utilizado para demonstrar o controle do posicionamento do arquivo durante a leitura.

Para encontrar os M menores valores, o programa utiliza a função 'val_min', que percorre o vetor e retorna o menor elemento disponível. Após encontrar cada mínimo, o valor correspondente no vetor original é substituído por

$$10^{300}.$$

Dessa forma, esse elemento deixa de ser considerado nas próximas buscas. O procedimento é repetido M vezes, e os valores encontrados são armazenados no vetor 'numeros_ordenados'.

Esse procedimento produz diretamente os M menores valores em ordem crescente. A estratégia pode ser interpretada como uma seleção sucessiva dos menores elementos: a cada etapa, o menor valor ainda disponível é identificado, armazenado e retirado das buscas seguintes.

3.3 Resultados Obtidos

Para o arquivo de entrada utilizado, o programa identificou corretamente

$$N = 500000$$

números. Em seguida, foi escolhido

$$M = 10.$$

O terminal apresentou os dez menores valores em ordem crescente:

Os 10 menores numeros em ordem crescente sao:

1.2451782822608948E-006
5.5236741900444031E-006
6.6380016505718231E-006
6.6407956182956696E-006
7.6387077569961548E-006
7.8030861914157867E-006
1.0204501450061798E-005
1.0691117495298386E-005
1.1510215699672699E-005
1.2444797903299332E-005

Os valores estão em ordem crescente, como esperado. O menor valor encontrado foi

$$1.2451782822608948 \times 10^{-6},$$

enquanto o décimo menor valor foi

$$1.2444797903299332 \times 10^{-5}.$$

O arquivo de saída também armazena inicialmente o valor de M e, nas linhas seguintes, os M valores selecionados. Assim, para o teste realizado, o arquivo contém $M = 10$ seguido pelos dez menores números do arquivo de entrada.

O resultado mostra que o algoritmo consegue selecionar somente a quantidade solicitada de menores elementos, sem precisar produzir uma ordenação completa dos 500000 números.

4 Problema 4: Série de Taylor para Cosseno

4.1 (a)

4.1.1 Descrição do Problema

O objetivo é calcular o valor de $\cos(x)$ utilizando sua expansão em série de Taylor, com precisão simples ('REAL*4'). O cálculo deve prosseguir até que o módulo do termo acrescentado à série seja menor ou igual a

$$\epsilon = 10^{-5}.$$

O resultado obtido pela série é comparado com o valor calculado pela função intrínseca 'COS' do FORTRAN, sendo também determinada a diferença absoluta entre os dois resultados.

4.1.2 Estratégia de Solução

A expansão de Taylor utilizada para o cosseno é

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!}.$$

O programa inicializa a soma com o primeiro termo da série,

$$\cos(x) \approx 1,$$

e, a cada iteração, calcula um novo termo

$$t_n = \frac{(-1)^n x^{2n}}{(2n)!}.$$

O laço é implementado utilizando a estrutura ‘GOTO’, disponível no FORTRAN-77. Após cada termo ser acrescentado à soma, seu módulo é comparado com ϵ . Enquanto

$$|t_n| > \epsilon,$$

um novo termo é calculado. Quando essa condição deixa de ser satisfeita, o processo é interrompido.

O fatorial $(2n)!$ é calculado separadamente pela função ‘fatorial’, que utiliza um laço ‘DO’ para multiplicar sucessivamente os inteiros de 2 até $2n$.

Após o cálculo da série, o programa utiliza a função intrínseca ‘COS(x)’ para obter um segundo valor e calcula a diferença absoluta:

$$\Delta = |\cos_{\text{Taylor}}(x) - \cos_{\text{FORTRAN}}(x)|.$$

4.1.3 Resultados Obtidos

Foi utilizado como teste o valor

$$x = 3.14 \text{ rad.}$$

A saída do programa, apresentada na mesma linha, foi:

-0.999998450 -0.999998748 2.98023224E-07

Os três valores correspondem, respectivamente, ao resultado obtido pela série de Taylor, ao resultado da função intrínseca ‘COS’ do FORTRAN e à diferença absoluta entre eles. Portanto,

$$\cos_{\text{Taylor}}(3.14) = -0.999998450,$$

$$\cos_{\text{FORTRAN}}(3.14) = -0.999998748$$

e

$$\Delta = 2.98023224 \times 10^{-7}.$$

A diferença obtida é significativamente menor que a precisão estabelecida para o critério de parada,

$$\Delta < 10^{-5}.$$

Isso indica que, para o valor testado, a soma dos termos da série forneceu uma aproximação próxima do valor calculado pela função intrínseca do FORTRAN.

4.2 (b)

4.2.1 Descrição do Problema

O objetivo é modificar o programa do item (a) para utilizar precisão dupla ('REAL*8') e investigar a influência do valor de ϵ na aproximação da série de Taylor para o cosseno. Para isso, são testados valores de ϵ progressivamente menores, comparando-se em cada etapa o resultado da série com o valor fornecido pela função intrínseca 'COS' do FORTRAN.

4.2.2 Estratégia de Solução

O programa utiliza variáveis 'REAL*8' tanto para o argumento x quanto para o cálculo da série, do fatorial e dos demais valores envolvidos. O valor inicial de ϵ é

$$\epsilon = 10^{-4}.$$

A cada iteração, esse valor é dividido por 10:

$$\epsilon \leftarrow \frac{\epsilon}{10}.$$

Para cada valor de ϵ , calcula-se novamente a série de Taylor e a diferença absoluta em relação à função intrínseca:

$$\Delta = |\cos_{\text{Taylor}}(x) - \cos_{\text{FORTRAN}}(x)|.$$

O processo continua enquanto $\Delta > 0$. Assim, o programa procura o valor de ϵ para o qual os dois resultados passam a coincidir numericamente na representação de precisão dupla.

A expansão utilizada permanece

$$\cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!}.$$

A função 'taylor_cos' adiciona termos enquanto

$$|t_n| > \epsilon,$$

enquanto a função 'fatorial' calcula $(2n)!$ por meio de um laço 'DO'.

4.2.3 Resultados Obtidos

Para o teste

$$x = 3.14 \text{ rad},$$

foram obtidos os seguintes resultados:

ϵ	$\cos_{\text{Taylor}}(x)$	$\cos_{\text{FORTRAN}}(x)$	Δ
10^{-5}	-0.99999859769260335	-0.99999873172753950	$1.3403493615 \times 10^{-7}$
10^{-6}	-0.99999873522108396	-0.99999873172753950	$3.4935444537 \times 10^{-9}$
10^{-7}	-0.99999873165272657	-0.99999873172753950	$7.4812933626 \times 10^{-11}$
10^{-8}	-0.99999873165272657	-0.99999873172753950	$7.4812933626 \times 10^{-11}$
10^{-9}	-0.99999873172887932	-0.99999873172753950	$1.3398171461 \times 10^{-12}$
10^{-10}	-0.99999873172887932	-0.99999873172753950	$1.3398171461 \times 10^{-12}$
10^{-11}	-0.99999873172751907	-0.99999873172753950	$2.0428103653 \times 10^{-14}$
10^{-12}	-0.99999873172753972	-0.99999873172753950	$2.2204460493 \times 10^{-16}$
10^{-13}	-0.99999873172753972	-0.99999873172753950	$2.2204460493 \times 10^{-16}$
10^{-14}	-0.99999873172753950	-0.99999873172753950	0

Observa-se que a redução de ϵ produz, em geral, uma aproximação progressivamente mais próxima do valor calculado pela função intrínseca. Para $\epsilon = 10^{-12}$, a diferença já é da ordem de

$$10^{-16},$$

atingindo o limite da precisão numérica observada nos cálculos em ‘REAL*8’.

Para $\epsilon = 10^{-14}$, os valores calculados pela série e pela função intrínseca coincidem na representação numérica utilizada:

$$\text{COSTaylor}(3.14) = \text{COSFORTRAN}(3.14) = -0.99999873172753950.$$

Nesse ponto,

$$\Delta = 0,$$

fazendo com que a condição do programa seja satisfeita e interrompendo o processo de redução de ϵ . O teste mostra, portanto, que a utilização de precisão dupla permite reduzir significativamente a diferença observada no item (a), embora a redução de ϵ não produza necessariamente uma mudança no resultado a cada etapa devido à precisão finita da representação numérica.

5 Problema 5: Sistemas Lineares e Regra de Cramer

5.1 (a)

5.1.1 Descrição do Problema

O objetivo é ler de um arquivo de entrada as permutações dos números $1, 2, \dots, N$, acompanhadas de suas respectivas paridades, e, a partir de cada permutação de N elementos, gerar as $N + 1$ permutações de $1, 2, \dots, N + 1$ obtidas pela inserção do elemento $N + 1$ em cada uma das posições possíveis.

Além da nova permutação, o programa deve calcular sua paridade e armazenar os resultados em um arquivo de saída. A paridade de uma permutação pode assumir os valores $+1$ ou -1 .

5.1.2 Estratégia de Solução

O programa recebe os nomes dos arquivos de entrada e saída e chama a subrotina ‘permutacoes_n1’, responsável pelo processamento das permutações.

Inicialmente, a primeira linha do arquivo é lida como uma cadeia de caracteres. A função ‘cardinalidade_numeros’ percorre essa cadeia e conta quantos números estão presentes. Como o último número corresponde à paridade da permutação, subtrai-se uma unidade para obter a ordem N da permutação.

Depois de determinar N , o arquivo é reposicionado no início utilizando ‘REWIND’. Cada linha é então lida contendo uma permutação de N elementos e sua paridade.

Para cada permutação, o programa insere o elemento $N + 1$ em cada uma das $N + 1$ posições possíveis. Essa operação é realizada pela subrotina ‘calcula_permutacao_n1’. Os elementos anteriores à posição de inserção são copiados diretamente, o elemento $N + 1$ é colocado na posição escolhida e os elementos restantes são deslocados uma posição para a direita.

A alteração da paridade é determinada pelo número de transposições necessárias para deslocar o elemento $N + 1$ da última posição até a posição escolhida. Se o elemento for inserido na posição i , são necessárias

$$N - i + 1$$

transposições. Como cada transposição altera o sinal da paridade, a nova paridade é calculada por

$$p_{N+1} = p_N (-1)^{N-i+1},$$

onde p_N é a paridade da permutação original.

Dessa forma, para cada uma das $N!$ permutações de ordem N , são geradas $N + 1$ novas permutações. O número total de permutações produzidas é, portanto,

$$N!(N + 1) = (N + 1)!.$$

5.1.3 Resultados Obtidos

No teste realizado, o programa identificou

$$N = 3.$$

Como existem

$$3! = 6$$

permutações de três elementos no arquivo de entrada e, para cada uma delas, são geradas

$$N + 1 = 4$$

novas permutações, o número esperado de permutações no arquivo de saída é

$$6 \times 4 = 24 = 4!.$$

O programa produziu exatamente 24 linhas, correspondentes às 24 permutações de quatro elementos. A saída começa com

```
4 1 2 3  -1
1 4 2 3   1
1 2 4 3  -1
1 2 3 4   1
```

Essas quatro linhas são obtidas a partir da primeira permutação de entrada,

$$(1, 2, 3),$$

cujas paridade é $+1$, pois ela possui zero inversões. O elemento 4 é inserido sucessivamente nas quatro posições possíveis.

Quando 4 é colocado na primeira posição, são necessárias três transposições para deslocá-lo da posição final até a primeira. Como três é ímpar, o sinal da paridade é invertido. Assim, a permutação

$$(4, 1, 2, 3)$$

possui paridade -1 , em concordância com a saída do programa.

Para as demais posições, o número de transposições necessárias diminui sucessivamente. Portanto, a sequência de paridades obtida é

$$-1, +1, -1, +1,$$

como mostrado na saída.

A partir das demais cinco permutações de três elementos, o mesmo procedimento é repetido. A saída completa contém 24 permutações, confirmando a relação

$$(N + 1)! = (3 + 1)! = 24.$$

5.2 (b)

5.2.1 Descrição do Problema

O objetivo é calcular o determinante de uma matriz real $N \times N$ lida de um arquivo de entrada, utilizando a fórmula de Leibniz. Para isso, são utilizadas as permutações de $1, 2, \dots, N$ e suas respectivas paridades, geradas pelo procedimento desenvolvido no item (a).

Para uma matriz A , o determinante é calculado pela expressão

$$\det(A) = \sum_{\sigma} \text{sgn}(\sigma) \prod_{i=1}^N a_{i,\sigma(i)},$$

em que a soma é realizada sobre todas as $N!$ permutações σ de $1, 2, \dots, N$. A paridade de cada permutação determina o sinal do termo correspondente.

O programa recebe como entrada um arquivo contendo os elementos da matriz e determina automaticamente sua dimensão N a partir da quantidade de elementos presentes na primeira linha. Em seguida, gera as permutações necessárias utilizando sucessivas aplicações da subrotina desenvolvida no item (a). Por fim, calcula o somatório da fórmula de Leibniz e fornece o determinante da matriz.

5.2.2 Estratégia de Solução

Inicialmente, o programa abre o arquivo de entrada e lê sua primeira linha como uma cadeia de caracteres. A função ‘cardinalidade_numeros’ conta a quantidade de números presentes nessa linha, permitindo determinar a dimensão N da matriz.

O arquivo é então reposicionado com ‘REWIND’ e os N^2 elementos são lidos e armazenados na matriz ‘matriz’. Como a função ‘determinante’ utiliza uma matriz de dimensão máxima 100×100 , os elementos são armazenados em ‘matriz(i,j)’.

Para obter as permutações de $1, 2, \dots, N$, o programa começa com a única permutação de um elemento,

$$(1),$$

cuja paridade é $+1$. Essa informação é armazenada inicialmente no arquivo temporário

1 1

A partir dessa permutação, a subrotina ‘permutacoes_n1’, desenvolvida no item (a), é utilizada sucessivamente. A cada chamada, as permutações de k elementos são utilizadas

para gerar as permutações de $k + 1$ elementos. Dois arquivos temporários são alternados entre entrada e saída para evitar a criação de um arquivo diferente para cada etapa.

Quando são obtidas todas as permutações de N elementos, o arquivo correspondente é aberto e suas linhas são lidas uma a uma. Cada linha contém uma permutação

$$(p_1, p_2, \dots, p_N)$$

e sua respectiva paridade p , com $p = +1$ ou $p = -1$.

Para cada permutação, o programa calcula o produto

$$A(1, p_1)A(2, p_2) \cdots A(N, p_N)$$

por meio de um laço sobre as linhas da matriz. O produto é então multiplicado pela paridade da permutação e adicionado ao acumulador ‘determinante’:

$$\det(A) = \det(A) + p \prod_{j=1}^N A(j, p_j).$$

Após a leitura de todas as $N!$ permutações, o valor acumulado corresponde ao determinante da matriz segundo a fórmula de Leibniz.

Para o caso particular $N = 1$, não é necessário realizar nenhuma geração adicional, pois a única permutação (1) já é a permutação final. Esse caso é tratado separadamente no programa.

5.2.3 Resultados Obtidos

Para testar o programa, foi utilizada a matriz

$$A = \begin{pmatrix} 2.0 & 1.0 & 3.0 \\ 4.0 & 0.0 & 1.0 \\ 5.0 & 2.0 & 2.0 \end{pmatrix}.$$

Como a matriz possui três colunas, o programa identifica

$$N = 3$$

e utiliza as

$$3! = 6$$

permutações de três elementos.

O programa produziu o seguinte resultado:

0 determinante eh 17.000000000000000

O valor pode ser verificado analiticamente pela expansão do determinante:

$$\begin{aligned} \det(A) &= 2 \begin{vmatrix} 0 & 1 \\ 2 & 2 \end{vmatrix} - 1 \begin{vmatrix} 4 & 1 \\ 5 & 2 \end{vmatrix} + 3 \begin{vmatrix} 4 & 0 \\ 5 & 2 \end{vmatrix} \\ &= 2(0 - 2) - (8 - 5) + 3(8 - 0) \\ &= -4 - 3 + 24 \\ &= 17. \end{aligned}$$

Portanto, o valor obtido numericamente pelo programa,

$$\boxed{\det(A) = 17},$$

coincide com o valor calculado analiticamente. Esse resultado valida, para a matriz de teste, a implementação da fórmula de Leibniz utilizando as permutações e suas respectivas paridades geradas pelo procedimento do item (a).

5.3 (c)

5.3.1 Descrição do Problema

O objetivo é resolver um sistema de equações lineares da forma

$$A\mathbf{x} = \mathbf{y},$$

em que A é uma matriz real não singular de ordem $N \times N$, $\mathbf{x}$ é o vetor de incógnitas e $\mathbf{y}$ é um vetor de ordem N . Os dados do sistema são fornecidos em um arquivo de entrada contendo a matriz A e o vetor $\mathbf{y}$.

A solução é obtida utilizando a Regra de Cramer. Para cada componente x_j do vetor solução, constrói-se uma matriz A_j substituindo a j -ésima coluna de A pelo vetor $\mathbf{y}$. A componente correspondente é então calculada por

$$x_j = \frac{\det(A_j)}{\det(A)}.$$

Os determinantes são calculados pela fórmula de Leibniz implementada no item (b), utilizando as permutações e suas respectivas paridades geradas no item (a). O programa foi testado para sistemas de ordem

$$N = 4, \quad N = 5, \quad N = 6.$$

5.3.2 Estratégia de Solução

O programa começa selecionando, para cada valor de N entre 4 e 6, o arquivo de entrada correspondente. A subrotina ‘solucao’ é então chamada para ler os dados e calcular o vetor solução.

Inicialmente, a primeira linha do arquivo é lida como uma cadeia de caracteres. A função ‘cardinalidade_numeros’, utilizada nos itens anteriores, determina a quantidade de elementos presentes nessa linha e, conseqüentemente, a ordem N da matriz.

Após o uso de ‘REWIND’, as N linhas seguintes são lidas e armazenadas na matriz A . Em seguida, a última linha do arquivo é lida como o vetor $\mathbf{y}$.

O primeiro passo para aplicar a Regra de Cramer é calcular o determinante da matriz original:

$$\det(A).$$

Para isso, a função ‘determinante’, desenvolvida no item (b), é utilizada. Como a matriz A é não singular, seu determinante é diferente de zero.

Em seguida, o programa calcula cada componente do vetor solução. Para cada $j = 1, \dots, N$, uma cópia da matriz A é criada na matriz auxiliar ‘matriz_aux’. A j -ésima coluna dessa matriz é então substituída pelos elementos do vetor $\mathbf{y}$, produzindo a matriz A_j :

$$A_j = \begin{pmatrix} a_{11} & \cdots & y_1 & \cdots & a_{1N} \\ a_{21} & \cdots & y_2 & \cdots & a_{2N} \\ \vdots & & \vdots & & \vdots \\ a_{N1} & \cdots & y_N & \cdots & a_{NN} \end{pmatrix}.$$

A matriz A_j é escrita em um arquivo temporário e seu determinante é calculado novamente pela função ‘determinante’. A componente correspondente da solução é então obtida pela expressão

$$x_j = \frac{\det(A_j)}{\det(A)}.$$

Esse procedimento é repetido para todas as N colunas. Ao final, o vetor $\mathbf{x}$ contém a solução do sistema.

Como a função ‘determinante’ utiliza as rotinas desenvolvidas nos itens anteriores, o cálculo de cada $\det(A_j)$ é realizado pela fórmula de Leibniz, envolvendo todas as $N!$ permutações de $1, 2, \dots, N$ e suas respectivas paridades.

5.3.3 Resultados Obtidos

Foram utilizados três sistemas lineares, de ordens $N = 4$, $N = 5$ e $N = 6$. As matrizes e os vetores dos sistemas foram fornecidos nos respectivos arquivos de entrada (tarefa-5c-entrada-4.in, tarefa-5c-entrada-5.in e tarefa-5c-entrada-6.in).

Para $N = 4$, o sistema utilizado foi

$$A = \begin{pmatrix} 4 & 1 & 2 & 0 \\ 1 & 5 & 1 & 2 \\ 2 & 1 & 6 & 1 \\ 0 & 2 & 1 & 4 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 10 \\ 13 \\ 15 \\ 8 \end{pmatrix}.$$

O programa encontrou

$$\mathbf{x} = \begin{pmatrix} 1.2245614035 \\ 1.7333333333 \\ 1.6842105263 \\ 0.7122807018 \end{pmatrix}.$$

A substituição desse vetor no sistema fornece

$$A\mathbf{x} = \begin{pmatrix} 10 \\ 13 \\ 15 \\ 8 \end{pmatrix} = \mathbf{y},$$

confirmando a solução.

Para $N = 5$, foi utilizado o sistema

$$A = \begin{pmatrix} 5 & 1 & 0 & 2 & 1 \\ 1 & 6 & 1 & 0 & 2 \\ 0 & 1 & 7 & 1 & 1 \\ 2 & 0 & 1 & 8 & 1 \\ 1 & 2 & 1 & 1 & 6 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 14 \\ 16 \\ 18 \\ 17 \\ 18 \end{pmatrix}.$$

A solução calculada foi

$$\mathbf{x} = \begin{pmatrix} 1.6582135813 \\ 1.5043674274 \\ 1.9354747816 \\ 1.2572555649 \\ 1.6900535362 \end{pmatrix}.$$

Substituindo o vetor calculado na equação matricial, obtém-se, numericamente,

$$A\mathbf{x} \approx \begin{pmatrix} 14 \\ 16 \\ 18 \\ 17 \\ 18 \end{pmatrix} = \mathbf{y}.$$

A pequena diferença numérica que pode surgir na operação de substituição é compatível com os erros de arredondamento da representação em ponto flutuante.

Para $N = 6$, foi utilizado o sistema

$$A = \begin{pmatrix} 6 & 1 & 0 & 1 & 2 & 0 \\ 1 & 7 & 1 & 0 & 1 & 2 \\ 0 & 1 & 8 & 1 & 0 & 1 \\ 1 & 0 & 1 & 9 & 1 & 0 \\ 2 & 1 & 0 & 1 & 8 & 1 \\ 0 & 2 & 1 & 0 & 1 & 7 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 11 \\ 17 \\ 19 \\ 21 \\ 20 \\ 18 \end{pmatrix}.$$

O programa encontrou

$$\mathbf{x} = \begin{pmatrix} 0.7296856700 \\ 1.3457788611 \\ 1.7619258650 \\ 1.8670982242 \\ 1.7045044475 \\ 1.6917159951 \end{pmatrix}.$$

A substituição do vetor na equação $A\mathbf{x} = \mathbf{y}$ fornece, numericamente,

$$A\mathbf{x} \approx \begin{pmatrix} 11 \\ 17 \\ 19 \\ 21 \\ 20 \\ 18 \end{pmatrix} = \mathbf{y}.$$

As verificações dos três sistemas mostram que os vetores calculados reproduzem os respectivos vetores $\mathbf{y}$ quando multiplicados pelas matrizes A . Portanto, os resultados obtidos são consistentes com os sistemas de entrada e confirmam a implementação da Regra de Cramer no programa.

6 Problema 6: Volume de Esferas em Múltiplas Dimensões

6.1 (a)

6.1.1 Descrição do Problema

O objetivo é implementar um programa em FORTRAN-77 capaz de calcular o volume de uma d -esfera de raio R para todas as dimensões inteiras de 0 até uma dimensão máxima $d_{\max}$. O volume de uma esfera em dimensão d é dado por

$$V_d(R) = \frac{\pi^{d/2}}{\Gamma(1 + \frac{d}{2})} R^d,$$

em que Γ representa a função Gama. O raio R e a dimensão máxima $d_{\max}$ são fornecidos pelo usuário durante a execução do programa.

Os valores calculados são apresentados na tela e armazenados em um arquivo de saída, permitindo a análise dos volumes para diferentes dimensões.

6.1.2 Estratégia de Solução

O programa principal solicita ao usuário a dimensão máxima $d_{\max}$ e o raio R . Em seguida, abre o arquivo de saída e utiliza um laço ‘do’ para percorrer todas as dimensões inteiras de 0 até $d_{\max}$. Para cada dimensão, a função ‘volume_d_esfera’ é chamada para calcular o volume correspondente.

O cálculo utiliza a expressão

$$V_d(R) = \frac{R^d \pi^{d/2}}{\Gamma(1 + \frac{d}{2})}.$$

A principal dificuldade está no cálculo da função Gama para os valores necessários. Como d é inteiro, o argumento da função Gama,

$$x = 1 + \frac{d}{2},$$

pode ser inteiro ou semi-inteiro. O programa trata esses dois casos separadamente. Quando x é inteiro, utiliza-se a relação

$$\Gamma(n) = (n - 1)!,$$

e a função ‘fatorial’ calcula o fatorial correspondente por meio de um laço ‘do’. Quando x é semi-inteiro, o programa parte da relação

$$\Gamma\left(\frac{1}{2}\right) = \sqrt{\pi}$$

e utiliza recursivamente a propriedade

$$\Gamma(x + 1) = x\Gamma(x).$$

A operação $n = \text{int}(x - 0.5)$ extrai a parte inteira de x , indicando quantas reduções unitárias devem ocorrer até atingir 0.5:

$$\Gamma(x) = (x-1)(x-2)\cdots(x-n)\Gamma\left(\frac{1}{2}\right) = (x-1)(x-2)\cdots(x-n)\sqrt{\pi}$$

Assim, para os argumentos semi-inteiros, a função Gama é obtida multiplicando sucessivamente os fatores apropriados por $\sqrt{\pi}$.

O programa utiliza a função ‘volume_d_esfera’ para reunir esses cálculos. A cada iteração do laço principal, o volume é escrito no arquivo de saída por meio do comando ‘write’ e também é exibido na tela.

6.1.3 Resultados Obtidos

Como teste, foram utilizados

$$d_{\max} = 10$$

e

$$R = 1.$$

O programa produziu os seguintes valores:

Dimensao	0 Volume	1.0000000000000000
Dimensao	1 Volume	2.0000000000000000
Dimensao	2 Volume	3.1415927410125732
Dimensao	3 Volume	4.1887903213500977
Dimensao	4 Volume	4.9348024751914465
Dimensao	5 Volume	5.2637893068708772
Dimensao	6 Volume	5.1677132114641093
Dimensao	7 Volume	4.7247663647671860
Dimensao	8 Volume	4.0587125781926048
Dimensao	9 Volume	3.2985092698962117
Dimensao	10 Volume	2.5501643947012624

Os primeiros valores podem ser comparados diretamente com os volumes conhecidos. Para $R = 1$ e $d = 0$, a expressão fornece

$$V_0 = 1,$$

que é um resultado geométrico interessante.

Para $d = 1$,

$$V_1 = 2,$$

que corresponde ao comprimento do intervalo $[-1, 1]$. Para $d = 2$,

$$V_2 = \pi,$$

correspondente à área do círculo unitário. Para $d = 3$,

$$V_3 = \frac{4\pi}{3},$$

correspondente ao volume da esfera usual de raio unitário.

Os valores calculados pelo programa para essas quatro dimensões são, respectivamente,

$$1.00000000000, \quad 2.00000000000, \quad 3.1415927410, \quad 4.1887903214.$$

Eles reproduzem os valores esperados para $d = 0$ e $d = 1$ e fornecem os valores correspondentes a π e $4\pi/3$ para $d = 2$ e $d = 3$. A pequena diferença em relação aos valores matemáticos de π e $4\pi/3$ decorre da precisão utilizada para a constante ‘pi’ no código.

A partir de $d = 4$, os volumes continuam sendo calculados pela mesma expressão. Para $R = 1$, observa-se que o volume aumenta até a dimensão $d = 5$ e passa a diminuir a partir de $d = 6$. Nesse teste, o maior valor obtido é aproximadamente

$$V_5 = 5.2637893069.$$

O comportamento observado mostra que o programa consegue calcular os volumes para diferentes dimensões utilizando a mesma expressão geral e o tratamento apropriado da função Gama para argumentos inteiros e semi-inteiros.

6.2 (b)

6.2.1 Descrição do Problema

O objetivo é utilizar a ferramenta XMGRACE para construir um gráfico comparando o volume V_d das d -esferas em função da dimensão d , para três valores do raio:

$$R = 0.9, \quad R = 1.0, \quad R = 1.1.$$

Os volumes devem ser calculados para as dimensões inteiras de $d = 0$ até $d = 25$. O gráfico permite observar como o volume das esferas varia com a dimensão e como essa variação depende do raio.

Para facilitar a interpretação dos resultados, o eixo horizontal foi identificado pela dimensão d e o eixo vertical pelo volume V_d . Também foram utilizadas legendas para distinguir as curvas correspondentes aos três valores de R .

6.2.2 Estratégia de Solução

O programa em FORTRAN-77 utiliza a função ‘volume_d_esfera’, desenvolvida no item (a), para calcular os volumes para cada dimensão entre 0 e 25. São utilizados três valores fixos do raio:

$$R_a = 0.9, \quad R_b = 1.0, \quad R_c = 1.1.$$

Para cada raio, o programa percorre as dimensões com um laço ‘do’ e escreve no arquivo correspondente dois valores por linha: a dimensão d e o volume calculado. Dessa forma, são gerados três arquivos de dados:

```
tarefa-6b-saida-1.dat
tarefa-6b-saida-2.dat
tarefa-6b-saida-3.dat
```

Cada arquivo contém os pares (d, V_d) para um dos valores de raio.

Os três arquivos foram importados para o XMGRACE e utilizados como três conjuntos de dados distintos. Cada conjunto foi associado a uma curva no mesmo gráfico, permitindo comparar diretamente os volumes para $R = 0.9$, $R = 1.0$ e $R = 1.1$.

O eixo horizontal foi configurado como a dimensão d , variando de 0 a 25, enquanto o eixo vertical representa o volume V_d . As três curvas receberam legendas indicando seus respectivos valores de R .

6.2.3 Resultados Obtidos

A Figura 1 apresenta os volumes calculados para as dimensões de 0 a 25 e para os três valores de raio considerados.

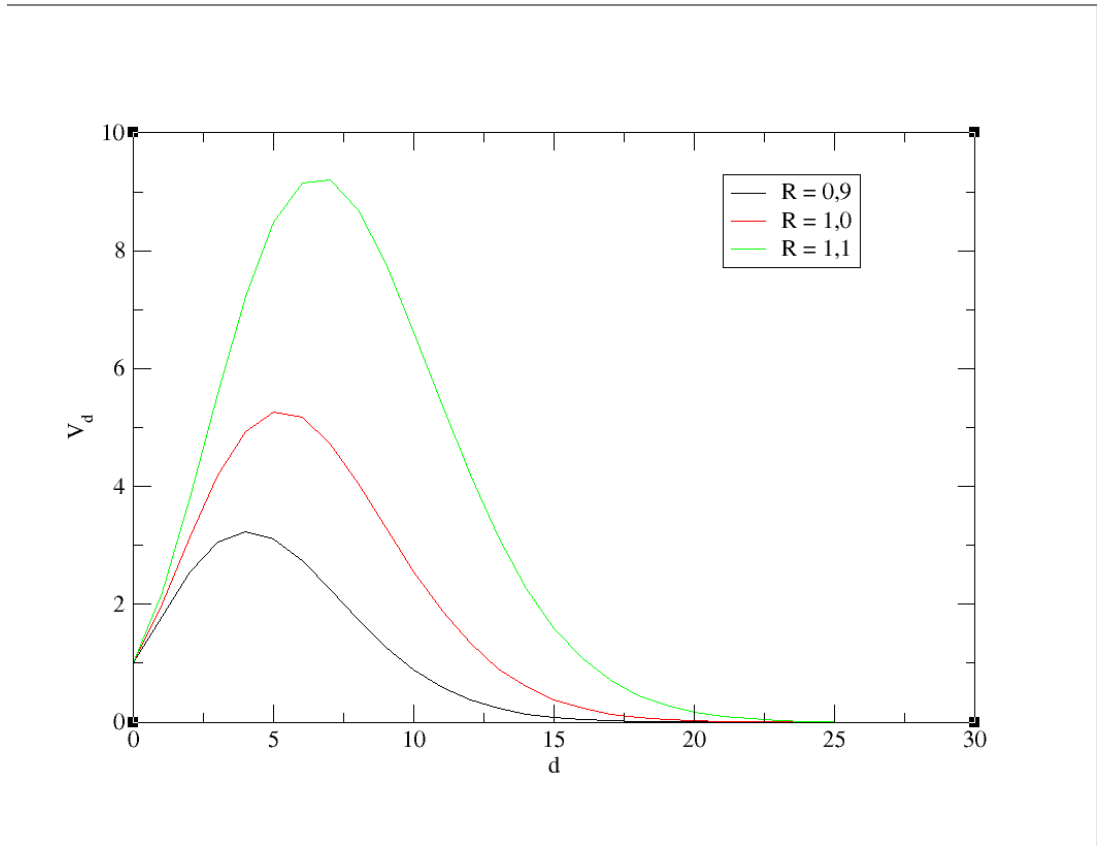


Figura 1: Volume da hiper-esfera V_d em função da dimensão d para raios $R = 0.9$, $R = 1.0$ e $R = 1.1$.

Para $d = 0$, as três curvas partem do mesmo valor,

$$V_0 = 1,$$

pois $R^0 = 1$ e $\Gamma(1) = 1$.

As curvas apresentam um ponto de máximo em dimensões intermediárias. Para $R = 1.0$, por exemplo, o maior valor ocorre em torno de $d = 5$, com

$$V_5 \approx 5.26.$$

O fator R^d faz com que diferenças entre os raios sejam amplificadas conforme a dimensão aumenta a partir da origem. Por isso, para dimensões pequenas, a curva correspondente a $R = 1.1$ cresce mais rapidamente que a de $R = 1.0$, enquanto a curva para $R = 0.9$ apresenta valores menores.

Apesar dessa diferença, as três curvas apresentam o mesmo comportamento assintótico: após atingirem seus respectivos máximos, os volumes diminuem e tendem a zero conforme a dimensão aumenta. Na região próxima de $d = 20$ a 25 , os valores das três curvas já são próximos de zero no gráfico.

Portanto, o gráfico obtido com o XMGRACE permite visualizar simultaneamente a existência de um máximo em dimensões intermediárias, a dependência do volume com o raio e o decaimento dos volumes para dimensões elevadas.

7 Problema 7: Relações Dimensionais e Número de Avogadro

7.1 Descrição do Problema

O problema 7 investiga a relação entre volumes em diferentes dimensões. Inicialmente, questiona-se quantas vezes o volume de um cubo de d dimensões com lado 1 m, cujo volume é 1 m^d , é maior que o volume de uma esfera de raio $R = 1 \text{ m}$ na mesma dimensão. Em seguida, analisa-se o comportamento dessa razão no limite $d \rightarrow \infty$. A segunda parte pede uma estimativa da ordem de grandeza do número de Avogadro em um espaço d -dimensional, considerando as escalas de volume de uma proteína ($1 \mu\text{m}^d$), de um átomo ($1 \text{ \AA}^d$) e de um volume macroscópico típico (1 mm^d).

7.2 Estratégia de Solução

Para a primeira parte, calcula-se a razão entre o volume do cubo e o volume da esfera. O volume do cubo de lado $L = 1 \text{ m}$ em d dimensões é

$$V_{\text{cubo}} = 1^d \text{ m}^d.$$

O volume da esfera de raio $R = 1 \text{ m}$ é obtido pela expressão para o volume de uma esfera em d dimensões. A razão η entre os dois volumes é então dada pela divisão do volume do cubo pelo volume da esfera. Para analisar o limite $d \rightarrow \infty$, considera-se o comportamento assintótico da Função Gama, usando a aproximação de Stirling.

Na segunda parte, convertem-se as unidades de volume para metros. Elevando os fatores de conversão à potência d , obtêm-se as ordens de grandeza dos volumes de um átomo e de um objeto macroscópico. O número de Avogadro em d dimensões é então estimado pela razão entre o volume macroscópico e o volume de um átomo.

7.3 Resultados Obtidos

7.3.1 (a)

A razão entre os volumes é

$$\eta = \frac{1}{\frac{\pi^{d/2}}{\Gamma(1+d/2)}} = \frac{\Gamma(1+d/2)}{\pi^{d/2}}.$$

Portanto, o volume do cubo é $\frac{\Gamma(1+d/2)}{\pi^{d/2}}$ vezes o volume da esfera. Para determinar o limite quando $d \rightarrow \infty$, usamos a aproximação de Stirling para a função Gama,

$$\Gamma(1+x) \sim \sqrt{2\pi x} \left(\frac{x}{e}\right)^x, \quad x \rightarrow \infty.$$

Tomando $x = d/2$, obtemos

$$\Gamma\left(1 + \frac{d}{2}\right) \sim \sqrt{\pi d} \left(\frac{d}{2e}\right)^{d/2}.$$

Logo,

$$\eta \sim \sqrt{\pi d} \left(\frac{d}{2\pi e}\right)^{d/2}.$$

Para verificar o limite, consideramos o logaritmo, que é uma função estritamente crescente:

$$\ln \eta \sim \frac{1}{2} \ln(\pi d) + \frac{d}{2} \ln\left(\frac{d}{2\pi e}\right).$$

Como $\ln\left(\frac{d}{2\pi e}\right) \rightarrow \infty$ quando $d \rightarrow \infty$, temos

$$\lim_{d \rightarrow \infty} \ln \eta = \infty.$$

Consequentemente,

$$\lim_{d \rightarrow \infty} \frac{\Gamma(1+d/2)}{\pi^{d/2}} = \infty.$$

Assim, a razão entre os volumes tende ao infinito.

7.3.2 (b)

Para estimar o número de Avogadro em d dimensões, considera-se a razão entre o volume macroscópico e o volume de um átomo. Como

$$1 \text{ Å} = 10^{-10} \text{ m} \quad \text{e} \quad 1 \text{ mm} = 10^{-3} \text{ m},$$

os volumes correspondentes são

$$V_{\text{átomo}} = (1 \text{ Å})^d = (10^{-10} \text{ m})^d = 10^{-10d} \text{ m}^d$$

e

$$V_{\text{macro}} = (1 \text{ mm})^d = (10^{-3} \text{ m})^d = 10^{-3d} \text{ m}^d.$$

Assim, o número de átomos que comporiam um objeto macroscópico seria aproximadamente

$$N_A = \frac{V_{\text{macro}}}{V_{\text{átomo}}} = \frac{10^{-3d} \text{ m}^d}{10^{-10d} \text{ m}^d} = 10^{7d}.$$

Portanto, a ordem de grandeza do número de Avogadro nesse espaço d -dimensional seria

$$N_A(d) \sim 10^{7d}. \quad (1)$$

Para $d = 3$, recupera-se

$$N_A(3) \sim 10^{21},$$

que corresponde à ordem de grandeza do número de átomos em um volume macroscópico de um milímetro cúbico.